

Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON Nov, 2025

Luxxcoin

Security Assessment



SAFEEDGES.IN

LuxxCoin Final Audit Report

Table of Contents

1. Executive Summary

2. Scope

3. Methodology

4. Severity Classification

5. Findings Overview

C-01 .Incorrect Round End-Time Calculation Allows Immediate Round Closure

H-01 .Permanent Token Lockout Due to Missing Withdrawal Mechanism

M-01 .Missing Deadline Enforcement Allows Late Reward Distribution

M-02 .Governance Can Modify Round Cadence, Breaking Immutable Schedule Guarantee

M-03 .Missing Round Start Time Validation Allows Premature Round Initialization and Schedule Bypass

M-04 .Missing Prefund Validation Allows Premature Round Initialization and Distribution

L-01 .Missing Duplicate Address Check in `distribute_batch` Allows Double Payouts

L-02 .Missing Automatic Round Finalization Allows Inconsistent Round State

6. Detailed Findings (with code, impact, and recommendations)

7. Conclusion

About Safe Edges

Safe Edges was established in 2023 by industry-renowned blockchain security expert. We are a trusted audit partner for leading Web3 protocols, decentralized finance (DeFi) platforms, NFT marketplaces, and Web2 enterprises.

Our mission is to secure the future of blockchain technology by providing comprehensive security solutions that protect digital assets and ensure the integrity of blockchain ecosystems.

With a team of top-ranked security researchers and former security engineers from major enterprises, we bring unparalleled expertise to every project we undertake.

Executive Summary

Item	Details
Scope	Review of the following repositories- https://github.com/Luxxcoin-team/Luxx-heroReward-program/tree/main/programs/hero-reward
Timeframe	November 2025
Methodology	Comprehensive manual code review, static analysis, threat-modeling, and best-practice compliance verification
Severity Summary	HIGH
Overall Risk	MEDIUM
Status	Fixed

Luxxcoin (\$LUX) is a Solana-based MemeFi project that turns meme culture into a rules-based, on-chain competition.

Luxxcoin (\$LUX) is a meme-powered Solana token that transforms HODLing into a recurring, live leaderboard game. Every two months, the top 50 wallets are rewarded with a massive share of 1.0 Billion \$LUX.

sbased on how much you hold, how long you've held, and how loyal you've been. There are no dev bags. No taxes. No VC allocation. Just pure community. Pure rewards. Pure meme competition

Techniques and Methods

Code quality checks

Best practices & documentation review

Gas efficiency analysis

Reentrancy & ERC compliance checks

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open	Resolved
Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	Partially Resolved
Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

ID	Title	Severity	Status
C-01	Incorrect Round End-Time Calculation AI-	CRITICAL	Fixed

	lows Immediate Round Closure		
H-01	Permanent Token Lock-out Due to Missing Withdrawal Mechanism	HIGH	Fixed
M-01	Missing Deadline Enforcement Allows Late Reward Distribution	MEDIUM	Acknowledged
M-02	Governance Can Modify Round Cadence, Breaking Immutable Schedule Guarantee	MEDIUM	Fixed
M-03	Missing Round Start Time Validation Allows Premature Round Initialization and Schedule Bypass	MEDIUM	Fixed
M-04	Missing Prefund Validation Allows Premature Round Initialization and Distribution	MEDIUM	Fixed
L-01	Missing Duplicate Address Check in distribute_batch Allows Double Payouts	LOW	Fixed
L-02	Missing Automatic Round Finalization Allows Inconsistent Round State	LOW	Fixed

Detailed Findings

[C-01] Incorrect Round End-Time Calculation Allows Immediate Round Closure

Summary

The `close_round` function uses an incorrect formula to compute a rounds `end_time`. Instead of calculating **(start of round + round duration)**, it computes **just the rounds start time**. This causes every round to appear ended the moment it begins. As a result, a round can be closed immediately after initialization—even within the same second—completely breaking the intended reward distribution timelines.

Vulnerability Details

The `close_round` function computes a rounds end time using the following formula:

```
round_end_time = tge_ts + (round - 1) * round_len_secs
```

This formula does **not** calculate the end time of the current round. Instead, it computes the **start time of the current round**, because:

What this calculates:

Round 1: $tge_ts + (1-1) * round_len = tge_ts + 0$ End time = Start of Round 1

Round 2: $tge_ts + (2-1) * round_len = tge_ts + 60 \text{ days}$ End time = Start of Round 2

Round 3: $tge_ts + (3-1) * round_len = tge_ts + 120 \text{ days}$ End time = Start of Round 3

As a result, **every rounds end time equals its start time**, meaning the round is considered over the moment it begins.

Thus, the condition:

```
let deadline_passed = current_time > round_end_time;
```

is immediately true as soon as the round is initialized—even one second after. This enables anyone to call `close_round` immediately after `initialize_round`, completely bypassing the intended full duration of the round.

Impact

Each round appears to end the moment it starts, allowing anyone to close it instantly.

Instant closure blocks `distribute_batch`, so no rewards are ever distributed for that round.

All rounds can be force-closed back-to-back, collapsing the entire 60-day schedule.

This destroys reward fairness, as participants across all rounds receive nothing.

The also creates a DoS vector, permanently disabling reward distribution for any prematurely closed round

Recommendation

Modify the `round_end_time` calculation with correct formula .

```
pub fn close_round(ctx: Context<CloseRound>, round: u8) -> Result<()> {
    require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
    require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
    let round_state = &mut ctx.accounts.round_state;
    require!(round_state.distribution_started, HeroRewardsError::DistributionNotStarted);
    require!(!round_state.distribution_finished, HeroRewardsError::DistributionFinished);
    // Check if cap reached or deadline passed
    let current_time = Clock::get()?.unix_timestamp;
    -- let round_end_time = ctx.accounts.state.tge_ts + (round as i64 - 1) *
    ctx.accounts.state.round_len_secs;
    ++ let round_end_time = ctx.accounts.state.tge_ts + (round as i64) * ctx.accounts.state.round_len_secs;
    let deadline_passed = current_time > round_end_time;
    let cap_reached = round_state.claimed_amount >= round_state.total;
    require!(cap_reached || deadline_passed, HeroRewardsError::RoundNotClosable);
    round_state.distribution_finished = true;
    emit!(RoundClosed {
        round,
        total_claimed: round_state.claimed_amount,
        cap_reached,
        deadline_passed,
    });
    Ok(())
}
```

Status Fixed

[H-01] Permanent Token Lockout Due to Missing Withdrawal Mechanism

Summary

The HeroRewards program receives **45B LUX tokens** via the `prefund()` step, storing them permanently in a PDA-owned vault. However, the program **does not include any function** that allows governance to **recover or sweep undistributed tokens**.

If any round is never initialized or distribution for a round is never executed, all tokens allocated to that round become **permanently stuck**, as only the publisher can trigger distribution and no fallback exists.

This creates a situation where **undistributed rewards can never be reclaimed**, even by governance .

Vulnerability Details

The vault ATA is owned by a PDA, and only the program can authorize transfers.

The program exposes **no function** to:

- Withdraw tokens back to governance,
- Sweep unused or expired round funds,

As a result:

- If the publisher never initializes or distributes a round, its tokens are locked forever.

- Any unused funds from skipped or failed rounds cannot be reclaimed.

Impact

This vulnerability can lead to a permanent loss of treasury tokens, with up to **45B LUX** becoming completely unrecoverable. Since governance has no mechanism to withdraw or reclaim stranded tokens from the PDA vault, any missed round, failed distribution, or publisher inactivity results in those funds being permanently locked.

Recommendation

Add a governance-only withdrawal / sweep function that allows the PDA vault to transfer unused or undistributed tokens back to the treasury or a designated authority .

Status Fixed

[M-01] Missing Deadline Enforcement Allows Late Reward Distribution

Summary

The `distribute_batch()` function does not enforce any deadline or round-expiry check. As a result, a publisher can continue distributing rewards for a round even after the rounds designated deadline has passed. This makes the round deadline meaningless and allows distributions to occur indefinitely until the round is manually closed. Even users may be forced to wait an indefinite amount of time for their rewards .

Vulnerability Details

The protocol defines rounds with deadlines, implying that distribution should only occur within a specific valid time window. However, `distribute_batch()` only checks:

```
distribution_started
```

```
distribution_finished
```

```
claimed_amount <= round_state.total
```

It **never checks** whether the round deadline has passed.

```
pub fn distribute_batch(
  ctx: Context<DistributeBatch>,
  round: u8,
  addresses: Vec<Pubkey>,
  amounts: Vec<u64>,
) -> Result<()> {
  require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
  require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
  require!(addresses.len() <= 20, HeroRewardsError::TooManyWinners); // Limit to 20
  per batch for safety
  require!(addresses.len() == amounts.len(), HeroRewardsError::InvalidAmount);
  require!(!addresses.is_empty(), HeroRewardsError::InvalidAmount);
  let round_state = &mut ctx.accounts.round_state;
```

```
require!(round_state.distribution_started, HeroRewardsError::DistributionNotStart-
ed);
require!(!round_state.distribution_finished, HeroRewardsError::DistributionFin-
ished);
// @audit-issue : Missing check for round deadline should not passed
// Calculate total amount to distribute
let total_amount: u64 = amounts.iter()
.try_fold(0u64, |acc, &amount| acc.checked_add(amount))
.ok_or(HeroRewardsError::Overflow)?;
// Check round cap
let current_claimed = round_state.claimed_amount
.checked_add(total_amount)
.ok_or(HeroRewardsError::Overflow)?;
require!(current_claimed <= round_state.total, HeroRewardsError::ExceedsRoundCap);
.....
}
```

This means:

Even when the rounds end timestamp is exceeded,

And even when previous rounds have correctly closed,

A publisher can still call `distribute_batch()` for any round where `distribution_finished` is not manually set to `true`.

Impact

Publisher can delay rewards indefinitely

Since the function having no deadline enforcement, a malicious publisher can intentionally delay distributions.

Rounds remain open far beyond the intended timeline.

Users may wait an indefinite amount of time for their rewards

Because the round never auto-finalizes, users could be forced to wait for long periods—minutes, hours, or even days—depending on publishers .

Recommendation

Add a strict `**deadline` check inside `distribute_batch()` .**

```
pub fn distribute_batch(
    ctx: Context<DistributeBatch>,
    round: u8,
    addresses: Vec<Pubkey>,
    amounts: Vec<u64>,
) -> Result<()> {
    require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
    require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
    require!(addresses.len() <= 20, HeroRewardsError::TooManyWinners); // Limit to 20
    per batch for safety
    require!(addresses.len() == amounts.len(), HeroRewardsError::InvalidAmount);
    require!(!addresses.is_empty(), HeroRewardsError::InvalidAmount);
    ++    let round_end_time = ctx.accounts.state.tge_ts + (round as i64) * ctx.ac-
    counts.state.round_len_secs;
    ++    require!(
    ++    Clock::get()?.unix_timestamp <= round_state.deadline,
    ++    HeroRewardsError::RoundExpired
    ++    );
    let round_state = &mut ctx.accounts.round_state;
    require!(round_state.distribution_started, HeroRewardsError::DistributionNotStart-
    ed);
    require!(!round_state.distribution_finished, HeroRewardsError::DistributionFin-
    ished);
    // Calculate total amount to distribute
    let total_amount: u64 = amounts.iter()
    .try_fold(0u64, |acc, &amount| acc.checked_add(amount))
    .ok_or(HeroRewardsError::Overflow)?;
    .....
}
```

Status Acknowledged

[M-02] Governance Can Modify Round Cadence, Breaking Immutable Schedule Guarantee

Summary

The protocol documentation explicitly states an "Immutable Schedule" with a "60-day cadence" for reward rounds. However, the `initialize` function accepts `round_len_secs` as a parameter that can be set by governance, allowing arbitrary modification of the round duration and breaking the promised immutable schedule.

Vulnerability Details

The code constants define `ROUND_LEN_DAYS: u64 = 60`, suggesting a fixed 60-day round duration. However, in the `initialize` function:

```
pub fn initialize(
    ctx: Context<Initialize>,
    tge_ts: i64,
    round_len_secs: i64,
) -> Result<()> {
    ....
    state.governance = ctx.accounts.governance.key();
    state.mint_lux = ctx.accounts.mint_lux.key();
    state.vault_lux = ctx.accounts.vault_lux.key();
    state.tge_ts = tge_ts;
    @-> state.round_len_secs = round_len_secs;
    state.total_rounds = TOTAL_ROUNDS;
    state.total_units = total_units;
    state.per_round_cap = per_round_cap;
    state.publisher = ctx.accounts.publisher.key();
    state.paused = false;
    state.prefunded = false;
    emit!(Initialized {
        tge_ts,
        round_len_secs,
        total_rounds: TOTAL_ROUNDS,
        total_units,
        publisher: ctx.accounts.publisher.key(),
    });
    ok(())
}
```

The `round_len_secs` parameter is stored directly without validation against the documented 60-day cadence. This allows governance to:

- Set rounds to any duration (shorter or longer than 60 days)

- Accelerate or delay token distribution arbitrarily

- Break the promised emission schedule

Impact

This allows governance to modify the round duration during initialization, breaking the documented 60-day immutable schedule guarantee .

Recommendation

Remove the `round_len_secs` parameter and use the constant value directly:

```
pub fn initialize(
  ctx: Context<Initialize>,
  tge_ts: i64,
  -- round_len_secs: i64,
) -> Result<()> {
  ...
  ++ let round_len_secs = (ROUND_LEN_DAYS * 24 * 60 * 60) as i64;
  state.governance = ctx.accounts.governance.key();
  state.mint_lux = ctx.accounts.mint_lux.key();
  state.vault_lux = ctx.accounts.vault_lux.key();
  state.tge_ts = tge_ts;
  ++ state.round_len_secs = round_len_secs;
  state.total_rounds = TOTAL_ROUNDS;
  state.total_units = total_units;
  state.per_round_cap = per_round_cap;
  state.publisher = ctx.accounts.publisher.key();
  state.paused = false;
  state.prefunded = false;
  emit!(Initialized {
    tge_ts,
    round_len_secs,
    total_rounds: TOTAL_ROUNDS,
    total_units,
    publisher: ctx.accounts.publisher.key(),
  });
  ok(())
}
```

Status Fixed

[M-03] Missing Round Start Time Validation Allows Premature Round Initialization and Schedule Bypass**

Summary

The `initialize_round()` function does not enforce that a round can only be initialized after its scheduled start time. While the protocol is designed to follow a strict time-based emission schedule derived from `tge_ts + (round-1) * round_len_secs`, the implementation having missing check for when rounds should actually start. This means the publisher can start any round whenever they want - even years early. This breaks the promised schedule and makes the timing rules useless.

Vulnerability Details

The expected start time for any round is:

```
round_start_time = tge_ts + (round - 1) * round_len_secs
```

However, `initialize_round()` never checks whether:

```
current_time >= round_start_time
```

This allows the publisher to initialize any round at any time, with no regard for the chronological order or intended timing.

Impact

Malicious publisher can initialize all 45 rounds immediately after program initialization, then distribute tokens at any time without waiting for the scheduled round start times.

When rounds are initialized prematurely, **higher-numbered rounds get disproportionately longer active durations** than intended:

(a). Round 1: If initialized immediately, remains active for ~60 days until its natural deadline

(b). Round 45: If initialized immediately, remains active for ~7.4 years until its natural deadline .

Loss of Fairness Across Rounds

Because higher rounds get massively extended windows:

Users in **later rounds** may receive rewards **much earlier** than expected

This results in **unequal and unpredictable reward timing**, even when users follow all rules.

Recommendation

Add time validation to `initialize_round`:

```
pub fn initialize_round(
  ctx: Context<InitializeRound>,
  round: u8,
  total: u64,
) -> Result<()> {
  require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
  require!(total as u128 == ctx.accounts.state.per_round_cap, HeroRewardsError::InvalidTotal);
  require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
  require!(ctx.accounts.state.prefunded, HeroRewardsError::NotPrefunded); // Also add prefund check
  // ADD TIME VALIDATION
  ++ let current_time = Clock::get()?.unix_timestamp;
  ++ let round_start_time = ctx.accounts.state.tge_ts + (round as i64 - 1) *
  ctx.accounts.state.round_len_secs;
  ++ require!(current_time >= round_start_time, HeroRewardsError::RoundNotStarted);
  let round_state = &mut ctx.accounts.round_state;
  require!(!round_state.distribution_started, HeroRewardsError::DistributionAlreadyStarted);
  round_state.round = round;
  round_state.total = total;
  round_state.distribution_started = true;
  round_state.distribution_finished = false;
  emit!(RoundInitialized {
    round,
    total,
    publisher: ctx.accounts.publisher.key(),
  });
  Ok(())
}
```



```
}
```

Status Fixed

[M-04] Missing Prefund Validation Allows Premature Round Initialization and Distribution**

Summary

`initialize_round` and `distribute_batch` are missing checks to ensure that `state.prefunded` is true. Without this validation, the publisher can call these functions before the `rew multisig` has funded the vault. As a result, rounds can be initialized before any tokens exist, and the system may attempt to distribute rewards from an empty vault.

Vulnerability Details

Missing Prefund Checks in Critical Functions

1.initialize_round

```
pub fn initialize_round(...) -> Result<()> {
  require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
  require!(total as u128 == ctx.accounts.state.per_round_cap, HeroRewardsError::InvalidTotal);
  require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
  // audit-issue : Missing require!(ctx.accounts.state.prefunded, HeroRewardsError::NotPrefunded);
  // ... proceeds to initialize round
}
```

This function allows new rounds to be initialized even when no tokens exist in the vault.

2distribute_batch

```
pub fn distribute_batch(...) -> Result<()> {
    require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
    require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
    // @audit-issue : Missing require!(ctx.accounts.state.prefunded, HeroRewardsError::Not-
    Prefunded);
    // ... attempts token transfers from empty vault
}
```

This results in batch distributions attempting SPL token transfers that are guaranteed to fail.

State Tracking Exists But Unused

```
pub struct HeroRewardsState {
    pub prefunded: bool, // Tracked but never validated in round operations
}
```

The program correctly records whether the vault was prefunded, but this value is **never used** to enforce correct protocol behavior.

Impact

Distribution attempts from empty vault will fail at the token program level

Publishers and winners pay transaction fees for operations that cannot succeed

Winners see distributions initiated but never receive tokens due to insufficient vault balance

Recommendation

Add the following check in both `initialize_round` and `distribute_batch`:

```
pub fn initialize_round(...) -> Result<()> {
    require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
    require!(total as u128 == ctx.accounts.state.per_round_cap, HeroRewardsError::Invalid-
    Total);
    require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
    ++ require!(ctx.accounts.state.prefunded, HeroRewardsError::NotPrefunded);
```

```
let round_state = &mut ctx.accounts.round_state;
require!(!round_state.distribution_started, HeroRewardsError::DistributionAlreadyS-
tarted);
.....
}
```

```
pub fn distribute_batch(
    ctx: Context<DistributeBatch>,
    round: u8,
    addresses: Vec<Pubkey>,
    amounts: Vec<u64>,
) -> Result<()> {
    require!(round > 0 && round <= TOTAL_ROUNDS, HeroRewardsError::InvalidRound);
    require!(!ctx.accounts.state.paused, HeroRewardsError::Paused);
    require!(addresses.len() <= 20, HeroRewardsError::TooManyWinners); // Limit to 20
    per batch for safety
    require!(addresses.len() == amounts.len(), HeroRewardsError::InvalidAmount);
    require!(!addresses.is_empty(), HeroRewardsError::InvalidAmount);
    ++ require!(ctx.accounts.state.prefunded, HeroRewardsError::NotPrefunded);
    let round_state = &mut ctx.accounts.round_state;
    require!(round_state.distribution_started, HeroRewardsError::DistributionNotStart-
ed);
    require!(!round_state.distribution_finished, HeroRewardsError::DistributionFin-
ished);
    .....
}
```

Status Fixed

[L-01] Missing Duplicate Address Check in `distribute_batch` Allows Double Payouts

Summary

The `distribute_batch` function does not verify that the list of recipient addresses contains unique entries. This allows a malicious publisher or misconfigured off-chain system to include the **same winner multiple times** in a single batch, causing that address to receive **multiple payouts** within one distribution call.

Vulnerability Details

Inside `distribute_batch`, the program iterates through:

```
pub fn distribute_batch(
    ctx: Context<DistributeBatch>,
    round: u8,
    addresses: Vec<Pubkey>,
    amounts: Vec<u64>,
) -> Result<()> {
    .....
    for (idx, (winner_address, amount)) in addresses.iter().zip(amounts.iter()).enumerate() {
        ...
    }
    .....
}
```

However, it **never checks** whether the `addresses` vector contains duplicates. If the same address is inserted more than once, the logic will:

Treat them as separate winners

Perform multiple token transfers

Inflate the total distribution for that batch (within cap)

Impact

This issue can cause **double or multiple payouts** to the same address within the same batch.

Recommendation

Add a uniqueness check before processing distribution .

Status Fixed

L-02. Missing Automatic Round Finalization Allows Inconsistent Round State

Summary

The `distribute_batch()` function never sets `round_state.distribution_finished = true`, even when the rounds distribution is fully completed. This allows a publisher to keep calling `distribute_batch()` indefinitely. While the round cap prevents over-distribution, the round is never formally closed, breaking expected round-lifecycle logic.

Vulnerability Details

Each round has two flags:

`distribution_started: bool`

`distribution_finished: bool`

`initialize_round()` sets:

```
pub fn initialize_round(
    ctx: Context<InitializeRound>,
    round: u8,
    total: u64,
) -> Result<()> {
    .....
    distribution_started = true;
    distribution_finished = false;
    .....
}
```

****But no where in `distribute_batch()` is `distribution_finished` set to true**, even when:**

```
pub fn distribute_batch(
    ctx: Context<DistributeBatch>,
    round: u8,
    addresses: Vec<Pubkey>,
    amounts: Vec<u64>,
) -> Result<()> {
    .....
    claimed_amount == total
    .....
}
```

meaning the round is effectively complete.

Only `close_round()` sets:

```
pub fn close_round(ctx: Context<CloseRound>, round: u8) -> Result<()> {  
    .....  
    distribution_finished = true;  
    .....  
}
```

If the publisher does not call `close_round()`, the round stays open forever.

The round remains in active distribution mode even after its cap is reached.

Impact

The round state becomes misleading:

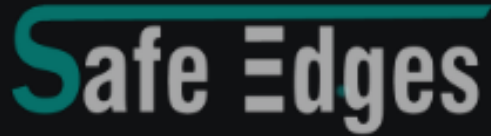
Rounds that are fully distributed still appear active.

Recommendation

Update distribution status when the cap is reached in the `distribute_batch()` function .

Status Fixed

Note - A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where We try to as many vulnerabilities as possible. we can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.



REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US



SAFEEDGES.IN